

COURSE CODE CSA0311

NAME Ranjitha R

COURSE NAME: Data Structure

REG No 192421285

C.O.4-AT-1 Comparative Analysis

Qn:1 Compare Best Case and Worst Case Time Complexity of sorting Algorithms:-

a) Best Case Time Complexity:-

Sorting Algorithm	Best Case
-------------------	-----------

Bubble Sort	$O(n)$
-------------	--------

Selection Sort	$O(n^2)$
----------------	----------

Insertion Sort	$O(n)$
----------------	--------

Merge Sort	$O(n \log n)$
------------	---------------

Quick Sort	$O(n \log n)$
------------	---------------

Heap Sort	$O(n \log n)$
-----------	---------------

b) Worst Case Time Complexity:-

Sorting Algorithm	Worst Case:-
-------------------	--------------

Bubble Sort	$O(n^2)$
-------------	----------

Selection Sort	$O(n^2)$
----------------	----------

Insertion Sort	$O(n^2)$
----------------	----------

Merge Sort	$O(n \log n)$
------------	---------------

Quick Sort	$O(n^2)$
------------	----------

Heap Sort

$O(n \log n)$

↳ practical Significance

Best case shows how fast an algorithm can work on favorable input.

Worst case shows the maximum time an algorithm may require.

d) Impact on Algorithm Selection :-

Insertion Sort : Good for small or nearly sorted arrays.

Merge sort : suitable when guaranteed $O(n \log n)$ performance is required.

Quick sort : Generally fast in practice but has $O(n^2)$ worst case.

Heap Sort : useful when $O(n \log n)$ worst case performance is required.

Qn : 2 Compare Recursive and Iterative implementation of Quick Sort..

a) Time Complexity

Implementation

Recursive Quick Sort

Best case

Worst case

$$O(n \log n)$$

$$O(n \log n)$$

Iterative Quick Sort

$$O(n \log n)$$

$$O(n \log n)$$

b) performance:-

Recursive Quick Sort:-

- Function call are automatically managed
- Simple and usually efficient
- Extra function call overhead is present.

Iterative Quick Sort:-

- Uses an explicit stack
- Avoid recursive function call overhead
- Gives more control over stack management.

c) Memory usage:-

Recursive

- Uses the system call stack
- Average space: $O(\log n)$
- Worst case $O(n)$

Iterative

- Uses an explicit stack

- Average space : $O(\log n)$

- Worst case : $O(n)$

d) Space Complexity:-

- * Recursive Quick Sort uses the recursion/call stack.

- * Iterative Quick Sort uses a manually created stack.

- * Both generally require $O(\log n)$ auxiliary space with balanced partitions.

- * Poor pivot selection can increase space to $O(n)$.

e) Ease of Implementation:-

Recursive Quick Sort

- * Easy to understand

- * Shorter code

- * Natural representation of the divide and conquer approach.

Iterative Quick Sort:-

- * More difficult to implement

- * Requires explicit stack management

- * Code is usually longer.

f. Risk of stack overflow:

Recursive Quick Sort.

- * higher risk of stack overflow
- * Especially when partition become highly unbalanced
- * Worst case recursion depth can become $O(n)$

Iterative Quick Sort:

- * lower risk of system call stack overflow
- + use an explicit stack instead.
- * Stack size can be controlled manually.